CLIENT DOCUMENTATION · 31 OCTOBER 2026

# DentaBook
# Client Handbook

Understanding, operating, and continuing your website.

Book your smile, know your cost.

> **Current release:** clinic discovery and account foundation. This handbook explains the delivered features, operating procedures, and remaining development phases.

# Contents


1. Project guide
2. User guide
3. Technical overview
4. Setup and deployment
5. Operations manual
6. Status and development plan
7. Testing and acceptance
8. API reference
9. Data and security
10. Handover checklist
11. Diagrams and workflows
12. Dependency versions
13. Branding, photography, and font notices
14. Delivery notes

# Project guide

## Purpose

DentaBook brings clinic information, treatment information, and account access into one website. A visitor can compare available clinic information and understand published treatment prices before contacting a clinic.

The longer-term product includes appointment requests, clinic review of those requests, deposits, appointment tracking, and completed-visit feedback. Those later steps are listed separately in the development plan because they are not available in the current release.

## Who uses the website

| Person | What the current release allows |
| --- | --- |
| Visitor | Browse clinics and treatments, view public profiles, search, and estimate costs without an account |
| Patient | Register, sign in, view account details, and view appointments already linked to that account |
| Clinic representative | Register a clinic, check review status, read review notes, and access the approved-clinic workspace |
| Platform administrator | Review clinic submissions and approve or reject pending registrations |

Each account is assigned a role. Registering as a patient cannot create a clinic or administrator account. Clinics appear publicly only after administrator approval. A clinic representative cannot enter the administrator workspace.

## What a visitor sees

The homepage introduces the service and provides access to clinic discovery, treatment information, the calculator, and patient account pages. Clinic listings come from approved records saved in the database. The site shows a clear empty message when no clinics have been published, and a separate error message if information cannot be loaded.

The calculator uses a selected clinic's saved service prices. It does not make a reservation, accept money, or guarantee a final treatment bill. The clinic confirms the treatment and final cost directly with the patient.

## Current delivery boundaries

This release supports discovery and account administration. A new clinic can register and be approved, but editing clinic descriptions, dentist records, prices, and schedules through the clinic dashboard is a later increment. Until that increment is delivered, an authorized operator must arrange data updates through a controlled maintenance process.

Patient appointment pages can display existing account-linked records. The website does not yet create new appointment requests or available time slots. A signed-in patient sees a clear notice where online booking is unavailable.

## Design approach

The interface uses DentaBook's supplied logo and a blue-and-white theme. Layouts begin with narrow phone screens, with additional arrangements for tablets and desktop computers. Forms stack vertically on phones. Form text is at least 16px, and buttons and navigation actions are designed for touch.

Operational information is presented as loading, empty, error, or unavailable states as appropriate. The application does not invent clinics, ratings, prices, or successful appointment confirmations.

## What this handoff contains

The source code, database initialization and migrations, configuration templates, automated tests, asset notices, and documentation are included. Live data, passwords, account recovery secrets, certificates, and private hosting credentials are transferred separately through the operator's agreed access process.

Use the user guide for step-by-step tasks and the development plan to review the remaining scope.

# User guide

For a visual walkthrough, see the patient, clinic, and administrator workflows.

## Browsing without an account

1. Open https://dentabook.forgroup.space.
2. Choose **Find a clinic** to open the directory.
3. Search by the clinic, service, dentist, or location information available in the saved records.
4. Open a clinic to view its public contact details, active services, and active dentist information.
5. Use **Cost calculator** to select a clinic and its published treatments.
6. Contact the clinic to confirm suitability, availability, and the final price.

If no clinics are listed, approved records have not been published yet. If an error message appears, use the retry action. An empty directory is not the same as a connection error.

## Creating a patient account

1. Open **Patient registration** at `/patient/register`.
2. Enter your name, email address, phone number, and password.
3. Confirm the same password and submit the form.
4. After the account-ready message, continue to patient sign-in.
5. Sign in with the email address and password you registered.

Passwords must contain at least 12 characters and no more than 72 UTF-8 bytes. Some symbols use more than one byte; an ordinary longer passphrase can meet the minimum without using complex symbols.

Patient registration does not grant clinic or administrator access. Password recovery and email verification are not yet implemented. If you forget your password, contact the designated website operator; there is no self-service reset link in this release.

## Using patient pages

**My account** shows the signed-in patient's saved information. **My appointments** shows appointments already associated with that account, or an empty message when none exist. Other patients' information is not available through these pages.

Opening a personal page while signed out leads to patient sign-in. After a successful sign-in, the website returns to the requested patient page. Online booking submission remains unavailable; signing in does not create an appointment.

## Registering a clinic

1. Open `/clinic/register` on the main website.
2. Enter the authorized account holder's name, account email, and password.
3. Enter the clinic's name, address, public phone, public email, and registration or license reference.
4. Confirm that you are authorized to register and manage the clinic.
5. Submit the registration.
6. Sign in at `/clinic/sign-in` to check the result.

The account email is used for sign-in. The public clinic email is contact information intended for the clinic's public listing. Submitting a registration does not automatically publish the clinic or approve its credentials.

## Clinic review results

| Status | Meaning and next step |
| --- | --- |
| Pending review | An administrator has not yet approved or rejected the submission |
| Approved | The clinic can appear in the public directory and the owner can access its workspace |
| Rejected | The clinic remains unpublished; read the review notes and contact the operator about correction |
| Suspended | The clinic workspace is unavailable while the suspension remains in place |

The approved-clinic workspace displays saved clinic information and available database counts. Editing services, dentist information, schedules, and appointments is planned work. The current administrator interface approves or rejects pending clinics; it does not provide a complete suspension-management workflow.

## Reviewing clinics as an administrator

1. Open https://dentabook-admin.forgroup.space/admin/sign-in.
2. Sign in with an administrator account provided through the operator setup process.
3. Open the list of pending clinic registrations.
4. Review the submitted details and reference information using the organization's review procedure.
5. Choose approve or reject and complete the confirmation step.
6. Provide a reason when rejecting a registration.
7. Confirm that the updated result appears in the appropriate list.

A review records the decision, reviewer, time, and notes. Approval is an application publishing decision; the software does not independently verify a professional license. There is no public administrator registration or default administrator password.

## Signing out and switching accounts

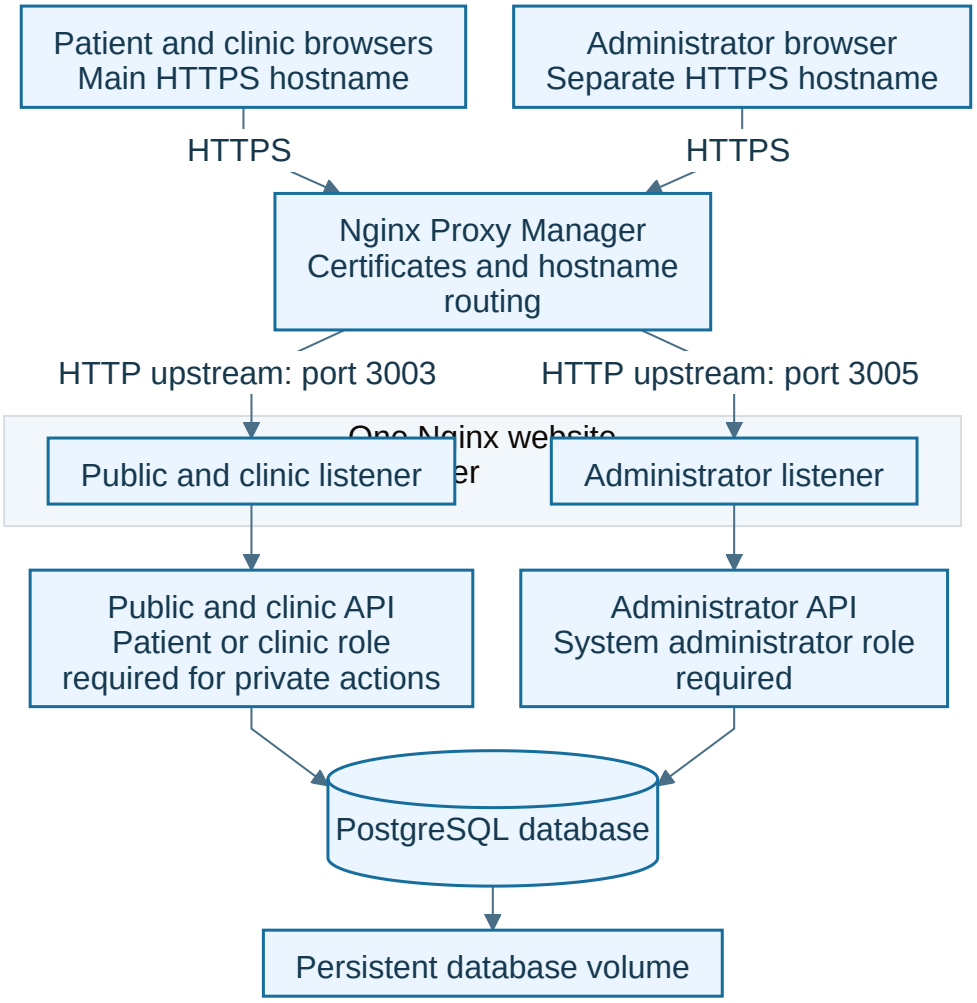
Choose **Sign out** before using a shared device or changing between patient and clinic accounts. Those two roles share the main website's browser session. Administrator access has its own hostname and session. Administrator sessions last one hour; patient and clinic sessions last eight hours unless revoked earlier.

If sign-in expires, sign in again. Changing a password or account role through an authorized maintenance process revokes existing sessions.

# Technical overview

## How the website is organized

DentaBook has three application layers: the browser interface, the server API, and the database. Production runs the API in two processes so administrator access is separated from the public and clinic site.




Current system architecture · Open the image at full size to zoom.

Open the diagram guide for the full-size architecture image, account/session sequence, and database relationships.

The browser presents forms and pages. The API validates submitted information and enforces account permissions. PostgreSQL stores the records. A browser cannot gain administrator access by changing a page link, form value, or request header.

## Technologies used

| Technology | Purpose |
| --- | --- |
| React 19 | Builds the website's reusable interface components |
| React Router 7 | Connects page addresses to the correct interface and handles navigation |
| Vite 6 | Runs the frontend development server and builds deployable website files |
| Standard CSS | Provides the mobile-first layout, theme, controls, and transitions |
| Lucide React | Supplies interface icons |
| Node.js 22 | Runs the server application and build tools |
| Express 4 | Organizes API requests and server middleware |
| node-postgres (`pg`) | Connects the server to PostgreSQL using pooled connections |
| PostgreSQL 16 | Stores accounts, clinic information, sessions, and related records |
| bcrypt 6 | Hashes passwords before they are stored |
| Nginx | Serves built website files and forwards API requests |
| Docker and Docker Compose | Package the services and define how they run together |
| Playwright | Checks browser behavior and responsive layouts |
| PGlite | Runs isolated PostgreSQL-compatible integration tests |

Exact installed package versions are recorded in both lockfiles and summarized in dependency versions. Package ranges in `package.json` describe allowed versions; `npm ci` installs the lockfile versions.

## Main source files

| Location | Responsibility |
| --- | --- |
| `client/src/App.jsx` | Website routes and public/clinic/admin page layouts |
| `client/src/pages/` | Public pages, registration, sign-in, and workspaces |
| `client/src/hooks/useAuth.jsx` | Browser session loading and access-related navigation |
| `client/src/hooks/useClinics.jsx` | Loading approved public clinic information |
| `client/src/api.js` | Requests from the browser to the same website's API |
| `client/src/index.css` | Branding, responsive layout, forms, and touch controls |
| `server/routes/` | Account, clinic directory, and workspace endpoints |
| `server/auth.js` | Session validation, password-related access controls, and role checks |

| Location | Responsibility |
| --- | --- |
| `server/portal.js` | Restricts which roles and endpoints each API process accepts |
| `server/database/` | Initial schema and ordered database migrations |
| `client/nginx-portals/` | Production public/admin listeners and forwarding rules |

## How information moves through the system

When a visitor opens the clinic directory, the browser requests `/api/clinics`. The API reads approved clinics and their active public services and dentist information. It returns data for the interface to display. If the query fails, the page shows a loading error rather than an invented clinic listing.

When a patient or clinic representative signs in, the server checks the password hash and required role. It creates a random session token, stores only its hash, and places the token in a protected browser cookie. Each private request is checked against the saved session. Authentication uses database-backed opaque sessions; it does not use JWT tokens.

When an administrator reviews a registration, the administrator API checks the session and role before changing a pending clinic's status. The change is saved with the reviewer and review information. Public directory queries only include approved clinics.

## Implementation approach

The application is organized into incremental capabilities: public discovery, account registration and permissions, clinic review, production separation, and testing. Interfaces use actual API data and explicit states for incomplete operations. Database changes are applied through versioned migration files rather than relying on manual changes to every installation.

The next increments extend clinic data management and scheduling before enabling appointment submission. The development plan explains this order and its acceptance criteria.

# Setup and deployment

This guide is for the person responsible for hosting. Routine users only need a browser and the website addresses in the user guide.

## Requirements

Use a server with Docker Engine and Docker Compose 2.24.4 or newer. Production requires the public and administrator hostnames to point to a reverse proxy, a valid certificate for each hostname, and HTTPS. Node.js 22 is needed on a workstation only when running development tools outside Docker.

Run a test installation on a separate machine from the live stack. The supplied container names are fixed; another copy on the same machine will conflict with the running containers unless the operator deliberately changes the configuration.

## Local evaluation

1. Copy `.env.example` to `.env` in the project root.
2. Replace the database password placeholder with a newly generated random password.
3. Set `NODE_ENV=development` and `CLIENT_URL=http://localhost:3000`.
4. Keep the production `COMPOSE_FILE` setting disabled for a combined local preview.
5. Confirm that ports 3000, 5000, and the chosen database diagnostic port are unused.
6. Run:

```
docker compose config --quiet
docker compose up -d --build --wait
docker compose ps
curl --fail http://localhost:3000/api/health
```

Open `http://localhost:3000`. Local patient, clinic, and administrator routes share that origin, with server-enforced roles. Local HTTP is for controlled evaluation; production uses HTTPS.

## Production configuration

On the intended production server, copy `.env.production.example` to `.env`. Preserve an existing production `.env` when updating a live installation. Supply a unique database password before first startup. On Linux, restrict the file with `chmod 600 .env`.

| Setting | Meaning |
| --- | --- |
| `COMPOSE_FILE` | Selects the base file and production override together |

| Setting | Meaning |
| --- | --- |
| `CLIENT_PORT=3003` | Host port for the patient/clinic website |
| `ADMIN_PORT=3005` | Host port for the administrator website |
| `CLIENT_URL` | Exact public HTTPS browser origin, with no path |
| `ADMIN_URL` | Exact administrator HTTPS origin, with no path |
| `DB_USER`, `DB_NAME`, `DB_PASSWORD` | Database connection settings; preserve these for an existing volume |
| `NODE_ENV=production` | Enables production operation and secure session cookies |
| `TRUST_PROXY` | Optional explicit trusted proxy addresses; leave blank until correctly configured |

Run all commands from the project root so Compose loads the intended `.env`:

```
docker compose config --quiet
docker compose build
docker compose up -d --wait
docker compose ps
```

The database initializes a new persistent volume on first startup. API startup applies pending migrations. An existing volume is reused on later deployments. Changing an environment password does not automatically change the password inside an existing database.

Production contains four services: the website, the public/clinic API, the administrator API, and PostgreSQL. The APIs and database have no published host ports. The website exposes ports 3003 and 3005 for the upstream reverse proxy.

## Nginx Proxy Manager

Create these two Proxy Hosts, substituting the production server's LAN address:

| Domain | Forward scheme | Forward host | Forward port |
| --- | --- | --- | --- |
| `dentabook.forgroup.space` | `http` | Production server LAN address | `3003` |
| `dentabook-admin.forgroup.space` | `http` | Production server LAN address | `3005` |

Install a valid certificate for each exact name and enable Force SSL. Forward the entire hostname; the application already handles `/api` forwarding. Do not create a separate clinic host. Clinic access uses `/clinic/sign-in` on the main website. Use Cloudflare Full (strict) encryption when the DNS records are proxied through Cloudflare.

The administrator root redirects to `/admin/sign-in`. Redirects are relative so they preserve the browser's HTTPS hostname or the published LAN port.

## First administrator

An operator creates the first real administrator interactively:

```
docker compose exec api npm run create-admin
```

Enter the intended person's name, email, and a private password. There are no default administrator credentials. Transfer access through the agreed private process, not through a source-code file or this documentation.

## Verify the installation

```
curl --fail https://dentabook.forgroup.space/api/health
curl --fail https://dentabook-admin.forgroup.space/api/health
curl --fail --location --head https://dentabook-admin.forgroup.space/
curl --fail --head https://dentabook.forgroup.space/clinic/sign-in
```

Both health responses should report `healthy` and a connected database. Administrator pages must return 404 on the public host; patient and clinic pages must return 404 on the administrator host. Then run the acceptance checks.

If the domains change, update the origins in `.env`, rebuild the client, recreate the API services, and update DNS/certificates. Public and clinic links are configured during the website build.

# Operations manual

## Routine checks

Use the correct project directory and production `.env`. Check `docker compose ps`, both HTTPS health endpoints, certificate renewal, available disk space, and the most recent verified backup. Agree an appropriate check schedule and responsible operator before relying on the system for daily operations.

```
docker compose ps
docker compose logs --tail=100 api admin_api client db
```

Logs can contain operational or account-related information. Share only the relevant, redacted excerpt when requesting support.

## Backing up the database

The persistent Docker volume keeps records when containers are replaced. It is not a separate backup. Create a private backup before every release or database maintenance operation, and on the schedule agreed with the project owner.

From the project root on a Linux host:

```
mkdir -p .local/backups
chmod 700 .local/backups
umask 077
docker compose exec -T db sh -c 'pg_dump -U "$POSTGRES_USER" "$POSTGRES_DB"' > ".local/backups/dentabook-$(date +%Y%m%d-
%H%M%S).sql"
```

Keep a protected copy outside the production server. Test recovery periodically in an isolated environment. Store the configuration and access needed for recovery separately from the database backup. This release does not configure scheduled backups, backup encryption, or retention automatically.

## Recovery procedure

1. Identify the last verified backup and the application version it belongs to.
2. Prepare an isolated PostgreSQL 16 database with the required database role and extension permissions.
3. Restore the SQL backup into a new, empty database using an authorized database operator's connection. A command such as `psql --set ON_ERROR_STOP=1 --dbname "$RESTORE_DATABASE_URL" --file backup.sql` must target that recovery database, not the live one.
4. Check account and clinic record counts, session behavior, directory data, and the relevant application workflows against the restored database.

5. Agree the production recovery window and data cutover before reconnecting the live application.
6. Repeat the health and acceptance checks after cutover.

The backup format is plain SQL. Restoring it into an already initialized database may conflict with existing tables or records; prepare an empty recovery database and stop on errors. Do not attempt a production restore by trial and error.

## Installing a new release

Confirm that the release has passed the agreed tests and read its release notes. Back up the database and retain the previous release and its configuration. For a clean client repository checkout, obtain the approved release using the agreed branch or tag, then run:

```
docker compose config --quiet
docker compose build
docker compose up -d --wait
docker compose exec -T client nginx -c /etc/nginx/portals/nginx.conf -t
docker compose exec -T client nginx -c /etc/nginx/portals/nginx.conf -s reload
```

Verify the two HTTPS health endpoints and the user journeys. Database migrations run at startup and may require a recovery plan beyond restarting an old image. The current project has no general automatic migration-down command.

Never use `docker compose down -v` for a routine update: it deletes the named database volume. `docker compose down` stops the stack without deleting that named volume. Keep production configuration independent from local testing.

## Common problems

| Symptom | What to check |
|---|---|
| Proxy shows 502 | Container status, production server address, correct upstream port, LAN reachability, and application logs |
| Certificate or hostname error | Exact Proxy Host name, DNS record, matching certificate, and renewal status |
| Sign-in form works but session does not persist | HTTPS, correct browser hostname, production origins, cookies, and API health |
| Registration returns a verification error | The browser is using the configured origin and requests carry the required application header |
| Clinic directory is empty | Whether clinics have been approved and active clinic service/dentist records exist |
| Clinic cannot open its workspace | Signed-in account role, clinic ownership, and approval status |
| Temporary too-many-attempts message | Respect the retry period; repeated requests may have reached the process's limit |

| Symptom | What to check |
| --- | --- |
| An administrator link fails on the main site | Use the separate administrator hostname; public-host admin routes are intentionally blocked |

Self-service password resets, MFA, automated email delivery, and provider payments are not available in this release. Escalate those requests to the designated maintainer using the approved maintenance process.

# Status and development plan

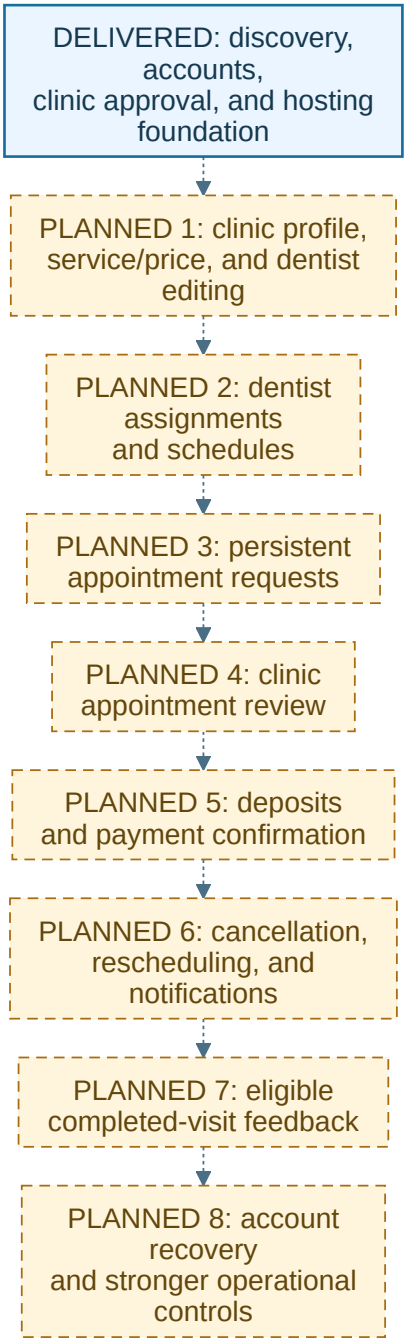
**Snapshot: 31 October 2026.** This document distinguishes delivered behavior from remaining scope. A database table or page address alone does not mean its complete workflow is available.

## Delivered in this snapshot

| Area | Available behavior |
|---|---|
| Public website | Homepage, treatment information, about page, and phone-first navigation |
| Clinic discovery | Approved-clinic directory, search, public clinic details, active services, and active dentist information |
| Cost estimates | Selected clinic's published prices; no charge or booking is created |
| Patient accounts | Registration, sign-in/out, account information, private-page protection |
| Patient appointments | Account-scoped reading of existing linked records, including an empty state |
| Clinic accounts | Registration, pending/rejected/suspended status information, review notes, and approved-clinic workspace |
| Administrator review | Separate sign-in and pending-clinic approval/rejection with recorded reviewer and notes |
| Access controls | Hashed passwords, database-backed sessions, role and ownership checks, origin checks, and separate administration |
| Hosting | Docker services, persistent storage, HTTPS proxy configuration, and deployment instructions |
| Verification | Automated backend and browser checks, including 320/390/768/1440px layouts |

## Current development position

The discovery and account foundation is delivered. The next proposed product increment is **clinic content management**. The remaining items below are planned or not implemented; they are not being presented as working features or as started tasks. Move an item to active development only when it has an assigned owner and work has begun. Update this document at each release.




Development roadmap — foundation delivered, remaining phases planned · Open the image at full size to zoom.

The planned booking diagram shows how the future steps fit together; it is not a live booking workflow.

## Remaining phases and completion criteria

| Order | Work | Prerequisite or client decision | Complete when |
|---|---|---|---|
| 1 | Clinic profile, service/price, and dentist editing | Confirm editable fields, publication rules, and credential handling | Approved clinic staff can save their own records; unauthorized edits are rejected; public views show the saved changes |
| 2 | Dentist-service assignments and schedules | Confirm working hours, appointment durations, timezone, and closure rules | Availability is derived from saved schedules and assignments, with invalid overlaps rejected |

| Order | Work | Prerequisite or client decision | Complete when |
|---|---|---|---|
| 3 | Persistent appointment requests | Finish schedule rules and patient intake fields | Signed-in patients can submit valid requests; concurrent conflicts are prevented in database transactions; requests survive reloads |
| 4 | Clinic appointment review | Agree acceptance/decline rules and responsibilities | Authorized clinics can review only their own requests and patients can see the saved outcomes |
| 5 | Deposits and payment confirmation | Select payment provider, deposit policy, refund process, and operator accounts | Provider-confirmed callbacks are verified, duplicate events are safe, amounts are checked on the server, and payment records reconcile |
| 6 | Cancellation, rescheduling, and notifications | Agree notice periods, allowed transitions, and delivery channels | Permitted changes are saved consistently and authorized recipients receive the correct notifications |
| 7 | Completed-visit feedback | Agree moderation and eligibility rules | Only eligible completed visits can receive feedback and publication follows the agreed policy |
| 8 | Account recovery and operational strengthening | Select email delivery, recovery procedures, backup ownership, and MFA scope | Recovery and verification are tested; scheduled backups and restore checks operate; required monitoring and controls are documented |

Each phase includes phone testing, access-control checks, real database persistence, failure handling, and updated documentation. Dates and effort estimates should be agreed after reviewing the phase's detailed requirements and dependencies.

## Decisions still needed

The next planning discussion should confirm who maintains clinic content, which credentials/documents may be stored, schedule and booking rules, deposit/refund policies, payment provider access, notification channels, backup retention, and the named operational owner. These decisions affect the design of the remaining workflows.

## Release workflow

1. Agree the phase scope and acceptance criteria.
2. Implement and test in the development environment.
3. Review the behavior with the project owner on phone and desktop.
4. Prepare an approved client release with updated status and release notes.
5. Back up production, deploy deliberately, and complete acceptance checks.
6. Record the deployed version and operational handover.

The client repository receives reviewed release snapshots. Changes in the development repository do not deploy to production or update client releases automatically.

# Testing and acceptance

## Automated backend checks

With Node.js 22 installed, from `server/`:

```
npm ci
node --test --test-concurrency=1 tests/*.test.js
```

These tests check database behavior, registration, approval, roles, record ownership, sessions, origin checks, cookie settings, and public/admin separation. Account integration tests use isolated PostgreSQL-compatible databases through PGlite. Test accounts belong to the test environment, not the live website.

## Frontend checks

From `client/`:

```
npm ci
npx playwright install chromium
npm run build
npm test
```

The standard browser suite starts its own local frontend and uses controlled API responses for interaction tests. Checks that require deployed hosts are skipped until their URLs are supplied. A skipped deployment check is not evidence that production was tested.

For the production public/admin journey checks:

```
PLAYWRIGHT_BASE_URL=https://dentabook.forgroup.space PLAYWRIGHT_ADMIN_URL=https://dentabook-admin.forgroup.space npm test --
portals.spec.js
```

Use the intended test hostnames if checking another installation; public-link assertions can be configured with `PLAYWRIGHT_PATIENT_PUBLIC_URL`. A different installed browser can be selected through `PLAYWRIGHT_CHANNEL` or `PLAYWRIGHT_CHROMIUM_EXECUTABLE_PATH`.

The deployed-listener checks read real APIs without creating live account records. They do not replace a controlled real-account registration and sign-in check before handover.

## Client review checklist

| Journey | Expected result |
| --- | --- |
| Browse signed out | Homepage, clinic discovery, treatments, and calculator are reachable |

| Journey | Expected result |
| --- | --- |
| Empty directory | A clear empty message, with no fabricated clinic records |
| Patient registration/sign-in | A saved patient account; private pages require its valid session |
| Clinic registration | A pending, unpublished clinic and staff account |
| Pending clinic access | Review status is visible; the approved workspace remains restricted |
| Administrator review | Only an administrator can approve/reject; a rejection requires a reason |
| Approved clinic access | The clinic owner sees its own saved information and counts |
| Wrong role | Patient, clinic, and administrator permissions remain distinct |
| Online booking | A clear unavailable notice until booking submission is implemented |
| Logout | The server session is revoked; private requests fail afterward |
| Main-host admin URL | Administration routes are blocked on the public hostname |
| Administrator root | The redirect preserves its HTTPS hostname or LAN port 3005 |

Use authorized test accounts in a test installation for workflows that create or change records. Use real production credentials only with the account owner's agreement, and remove temporary verification data through the operator's process.

## Screen sizes

Check a phone first. Review 320px, 390px, 768px, and 1440px widths. Pages must avoid horizontal overflow, text fields must be at least 16px, and buttons/navigation actions must have at least a 44 by 44px target. Review registration, sign-in, public navigation, empty/error states, and private workspace layouts.

Record the release, environment, date, checks run, results, and unresolved issues with each handover. Automated checks reduce regressions; they are not an independent penetration test or a promise that every browser condition has been examined.

# API reference

This appendix is for developers and operators. Browser users should use the normal website forms.

## Hosts and request rules

Public and clinic requests use `https://dentabook.forgroup.space/api`. Administrator requests use `https://dentabook-admin.forgroup.space/api`. Browser requests stay on the current host and use its session cookie.

State-changing requests use JSON, `Content-Type: application/json`, and `X-DentaBook-Request: 1`. The server rejects mismatched browser origins and cross-site request metadata. A valid session is still required where shown below. The configured API process rejects routes belonging to another production area with 404.

## Public and clinic endpoints

| Method | Path after `/api` | Access and purpose |
|---|---|---|
| GET | `/health` | Reports service and database readiness |
| GET | `/clinics` | Approved public clinic directory |
| GET | `/clinics/:id` | Approved clinic details with active public services/dentists |
| POST | `/auth/register-patient` | Validated patient registration; the role is assigned by the server |
| POST | `/auth/patient/login` | Patient-only sign-in |
| POST | `/auth/register-clinic` | Transactional clinic/staff registration, initially pending review |
| POST | `/auth/clinic/login` | Clinic-staff-only sign-in |
| POST | `/auth/login` | Existing clinic-only compatibility alias |
| GET | `/auth/me` | Valid session; current account and owned clinic information when applicable |
| POST | `/auth/logout` | Revokes the presented session and clears its cookie |
| GET | `/workspaces/patient/appointments` | Patient session; latest 50 linked appointments |
| GET | `/workspaces/clinic` | Approved clinic owner's saved clinic details and counts |

Patient registration accepts `name`, `email`, `phone`, and `password`. Clinic registration accepts `ownerName`, `email`, `password`, `clinicName`, `address`, `phone`, `contactEmail`, `registrationReference`, and `consent: true`. Client-side password confirmation is not a stored field. Unknown role/status fields cannot select elevated privileges.

## Administrator endpoints

The administrator host also provides `/health`, the public clinic read endpoints, and its own `/auth/me` and `/auth/logout` session operations.

| Method | Path after `/api` | Access and purpose |
|---|---|---|
| POST | `/auth/admin/login` | System-administrator-only sign-in |
| GET | `/workspaces/admin/clinics?status=pending_review` | Administrator session; registrations in the chosen status |
| POST | `/workspaces/admin/clinics/:id/review` | Administrator session; review a pending registration |

The review body uses `decision: "approved"` or `decision: "rejected"`, with optional `notes`. Rejection requires nonempty notes. Notes are limited to 2,000 characters. A previously reviewed registration returns a conflict instead of being reviewed again as pending.

## Responses and errors

Successful endpoints return JSON, except logout returns HTTP 204. Sign-in sets a protected cookie; it does not return a bearer token in the response body. Private session/workspace responses use `Cache-Control: no-store`.

| Status | Meaning |
|---|---|
| 200 / 201 | Successful read/action or new registration |
| 204 | Logout completed |
| 400 | Invalid input; registration may include field-specific errors |
| 401 | Sign-in required, expired session, or incorrect login credentials/role |
| 403 | Role, approval, or origin restrictions deny the request |
| 404 | Unknown, unpublished, or wrong-portal route/resource |
| 409 | Duplicate registration or review conflict |
| 429 | Temporary request-rate limit; follow `Retry-After` |
| 503 | Service or database temporarily unavailable |

Most errors use an `error` message, with `fields` where relevant. `/health` uses its status/database readiness structure. No appointment-creation, payment, scheduling, or password-reset API is delivered in this snapshot.

# Data and security

The diagram guide includes the current session-check sequence and simplified database relationships, with future management workflows labeled separately.

## Stored information

| Record group | Current use |
| --- | --- |
| Users | Name, email, phone where collected, password hash, and assigned role |
| Clinics | Ownership, clinic contact details, description, approval status, submitted reference, and review information |
| Dentists and services | Active public profile information and clinic service prices when records are provisioned |
| Sessions | Token hash, user, sign-in role, creation time, and expiry |
| Schema migrations | Database updates already applied |
| Appointments and appointment services | Existing patient-linked records can be read; website booking creation is not implemented |
| Schedules, assignments, payments, reviews | Schema foundations for later workflows; their complete management features are not available |

Database relationships link clinics to their owners, dentists/services to clinics, and appointments to patients and clinics. The API uses the session's account identity for private data rather than trusting a patient or clinic identifier supplied by the browser.

## Account protection

Passwords are hashed with bcrypt at cost 12. They are not stored as readable passwords. Sign-in issues a random 256-bit opaque session token; only its SHA-256 hash is stored in the database. The token is sent in a protected browser cookie and checked against a valid database session on private requests.

Sessions are bound to the account's role. Password-hash or role changes revoke existing sessions through a database trigger. Re-login rotates the presented browser session, logout revokes it, and expiration is enforced on the server. Administrator sessions last one hour; patient/clinic sessions last eight hours.

Production uses separate public and administrator host-only cookies with `Secure`, `HttpOnly`, and `SameSite=Strict`. The `__Host-` cookie prefix, a root path, and the absence of a Domain attribute limit cookie scope. Cookies are not isolated by port alone; separate hostnames and cookie names provide administrator session separation.

## Server and browser controls

The APIs validate inputs, enforce roles and clinic approval, parameterize data queries, check request origins, and reject unsupported portal endpoints. Nginx sends content-type, framing, referrer, permissions, and content-security headers. Private workspace and session responses are not cached.

API processes run as a non-root account with a read-only container filesystem, temporary storage, removed Linux capabilities, and no privilege escalation. PostgreSQL uses an internal Docker network. The production configuration publishes website ports 3003 and 3005, not direct API/database ports.

## Limits and continuing work

HTTPS protects browser-to-proxy traffic when DNS, certificates, and redirects are configured correctly. The proxy-to-application LAN hop and application-to-database Docker hop are not encrypted by the current setup. Database-volume and backup encryption are not automatically configured.

MFA, self-service password recovery, email verification, durable shared request throttling, and a full security audit trail are future work. Current request limits are stored in each API process's memory and reset with that process. The project has automated access-control tests; it has not undergone an independent penetration assessment.

Operators with host or database access can access stored records. Control access to hosting, database backups, source repositories, and configuration separately. Agree data retention, account support, and recovery responsibilities with the project owner before collecting additional information in later features.

## Configuration and backups

Actual database passwords belong in a private `.env`, not in source control. Keep backup files private and outside release archives. Preserve included dependency and asset notices. Transfer access through the agreed private process and rotate credentials when access responsibility changes.

# Handover checklist

Use this list when reviewing a client release. Record the named owners and acceptance date in the agreed project record; do not place passwords in this file.

## Delivery review

- ☐ Confirm the delivered release and source-code archive.
- ☐ Read the current feature/status table and agree which remaining phases are in scope.
- ☐ Confirm that the patient and clinic host is accessible over HTTPS.
- ☐ Confirm that the separate administrator host is accessible over HTTPS.
- ☐ Review the website on phone, tablet, and desktop.
- ☐ Complete the agreed account and clinic-review acceptance checks.
- ☐ Review known limitations, especially booking, payments, password recovery, and operational automation.

## Access and responsibility

- ☐ Assign the website owner and day-to-day administrator.
- ☐ Assign the hosting, DNS, certificate, and database operator.
- ☐ Provide the first intended administrator account through a private process.
- ☐ Confirm repository access and who approves future releases.
- ☐ Transfer necessary hosting and DNS access privately.
- ☐ Agree where problems and change requests are reported.
- ☐ Agree backup frequency, retention, protection, and who verifies recovery.
- ☐ Confirm that accounts belonging to previous operators are removed or reviewed when responsibility changes.

## Ongoing development

Keep this client repository as a reviewed release record. Development takes place in its separate working repository and environment. An approved delivery copies the application and updated documentation into this repository without bringing unrelated internal records into the release.

For each later release, update `CHANGELOG.md`, the status/development plan, any changed user/API instructions, and the verification record. The client should be able to identify the delivered behavior and outstanding work from those documents without reading development discussions.

# Review record

| Item | To be recorded by the responsible parties |
| --- | --- |
| Release accepted | Branch, tag, or commit of the client release |
| Acceptance date | Date the review is completed |
| Accepted by | Authorized client representative |
| Operations owner | Hosting/database responsible person |
| Support contact | Agreed reporting channel |
| Open items | Links to the agreed remaining-work list |
| Next review | Agreed review point |

# Diagrams and workflows

These diagrams explain the delivered application and the remaining development plan. **Current** means available behavior or an existing schema/process. **Planned** means not implemented. A schema diagram describes stored relationships, not a promise that the corresponding management screens exist.

Blue boxes represent current behavior. Dashed amber boxes explicitly marked PLANNED describe future work. Grey boxes mark an unavailable operation or a boundary. Labels preserve these meanings when printed without color.

On a phone, open an SVG at full size to zoom. In the HTML handbook, wide diagrams have their own scroll area. All diagrams are included as vector images for clear printing, with editable Mermaid sources alongside them.

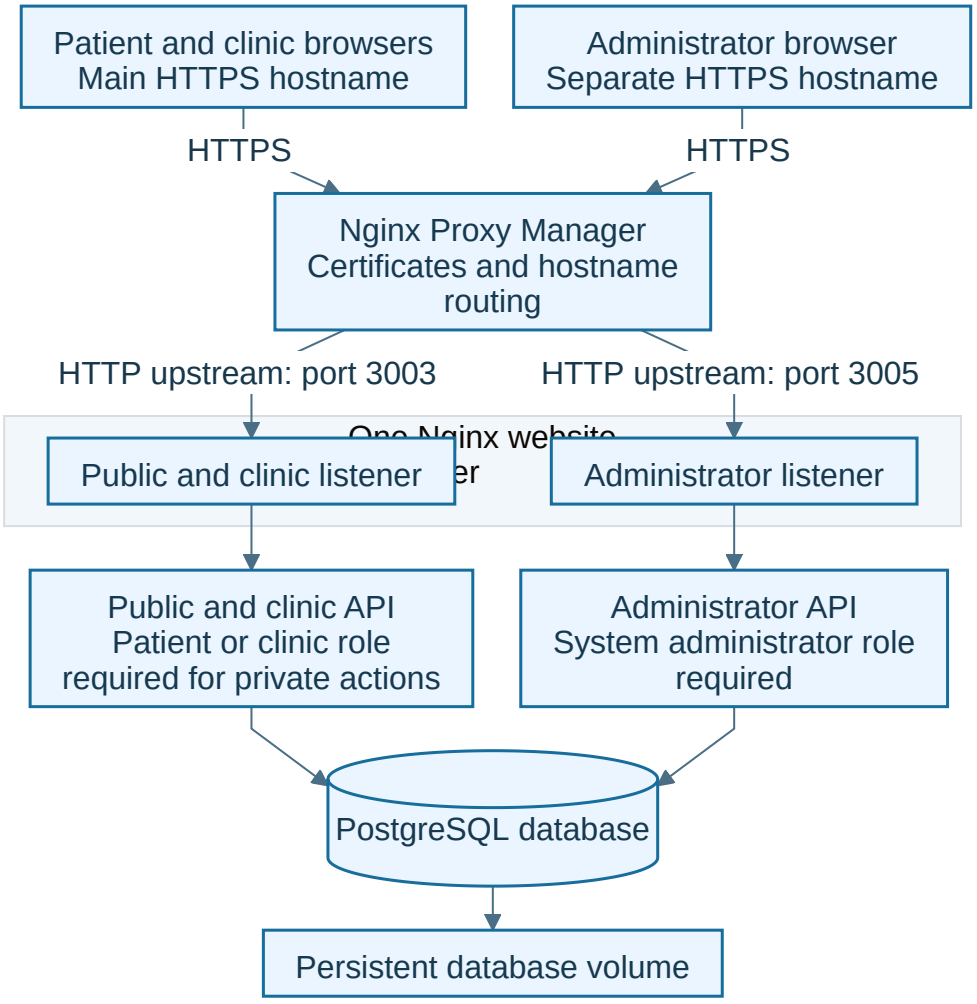
## Diagram index

| Diagram | Status | Main question answered |
|---|---|---|
| System architecture | Current deployment | The public hostname is dentabook.forgroup.space; administration uses dentabook-admin.forgroup.space |
| Patient browsing and account workflow | Current workflow | Visitors do not need an account to browse or estimate costs |
| Clinic registration and access | Current workflow | Registration and staff-account creation are one database transaction |
| Administrator review process | Current workflow | The review API permits only pending-to-approved or pending-to-rejected changes |
| Sign-in and session creation | Current sequence | Only the hash of the random session token is stored |
| Private request permissions | Current sequence | A valid session still needs role, ownership, and clinic-approval checks |
| Accounts, clinics, and public data | Current schema | This is a simplified view of actual foreign keys, not every column |
| Appointment and payment data foundations | Existing schema; management workflows planned | The tables already exist, but they do not establish a working booking or payment feature |
| Future appointment workflow | Planned; not a live booking process | Dashed amber steps are proposed development, not available features |
| Reviewed releases and deployment | Current manual operating process | This is an operator-led process, not automatic deployment |

| Diagram | Status | Main question answered |
| --- | --- | --- |
| Remaining development phases | Foundation delivered; next phases planned | The order follows the development plan |
| Published-price cost estimation | Current workflow | The calculator reads published database prices but performs the informational total in the browser |

## System architecture

**Status: Current deployment.**



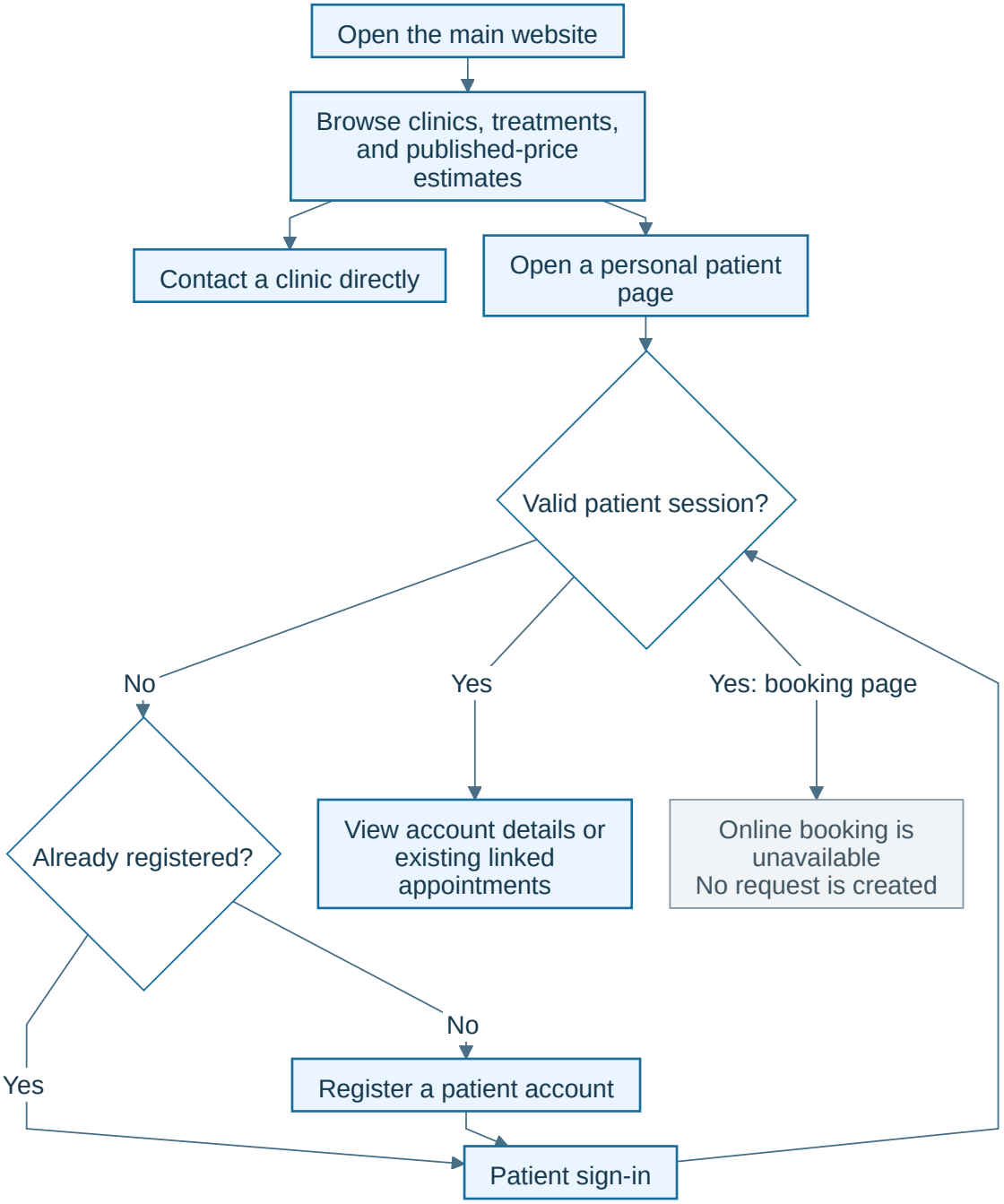

System architecture — Current deployment · Open the image at full size to zoom.

The public hostname is dentabook.forgroup.space; administration uses dentabook-admin.forgroup.space. Both APIs share the production database. HTTPS terminates at the external proxy; the upstream LAN links use HTTP. The APIs and database have no published production ports.

Open full-size SVG · Editable Mermaid source

## Patient browsing and account workflow

**Status: Current workflow.**



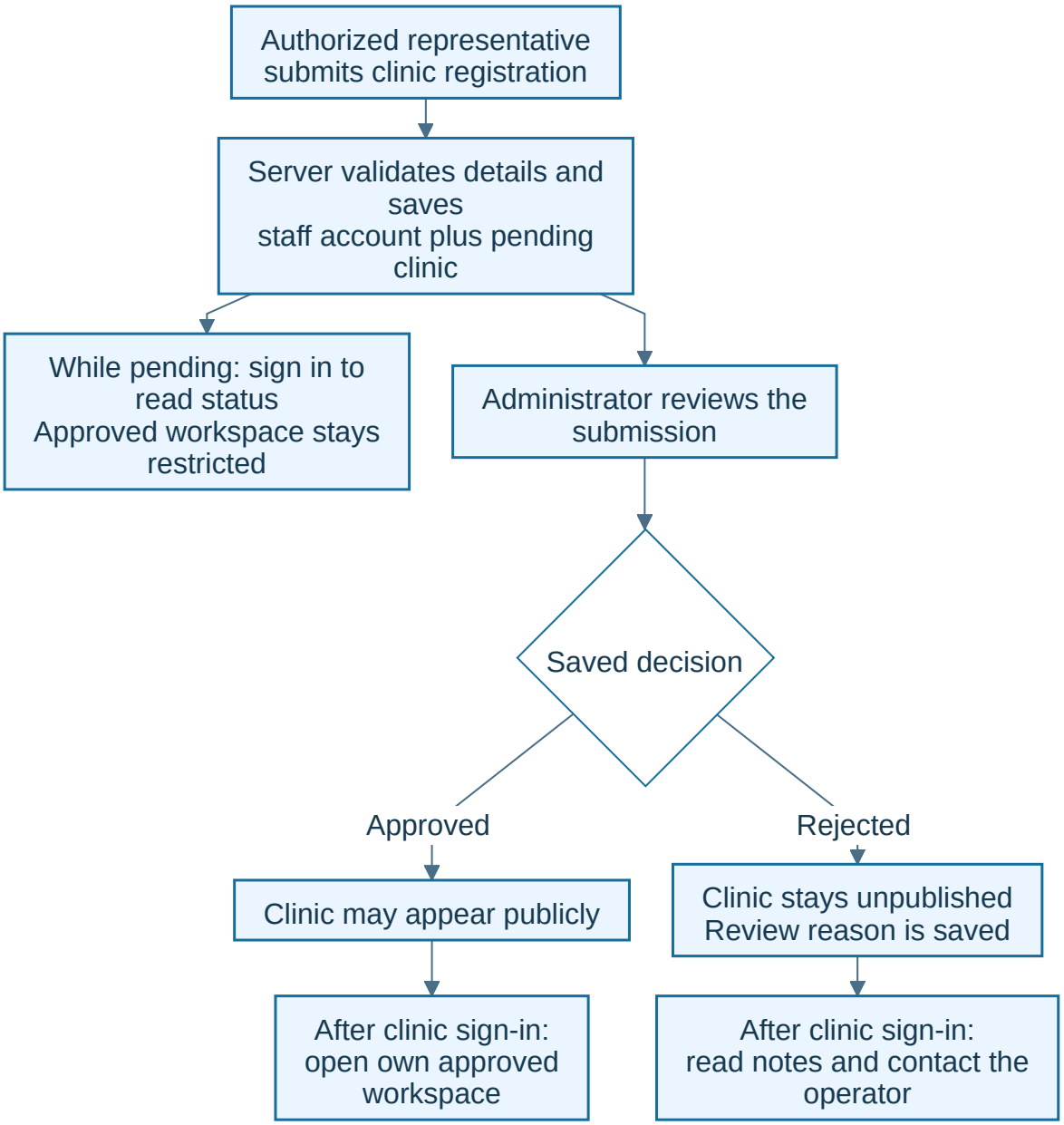

Patient browsing and account workflow — Current workflow · Open the image at full size to zoom.

Visitors do not need an account to browse or estimate costs. Personal pages require a server-validated patient session. Existing appointment records can be read; new booking submission is not implemented.

Open full-size SVG · Editable Mermaid source

## Clinic registration and access

**Status: Current workflow.**



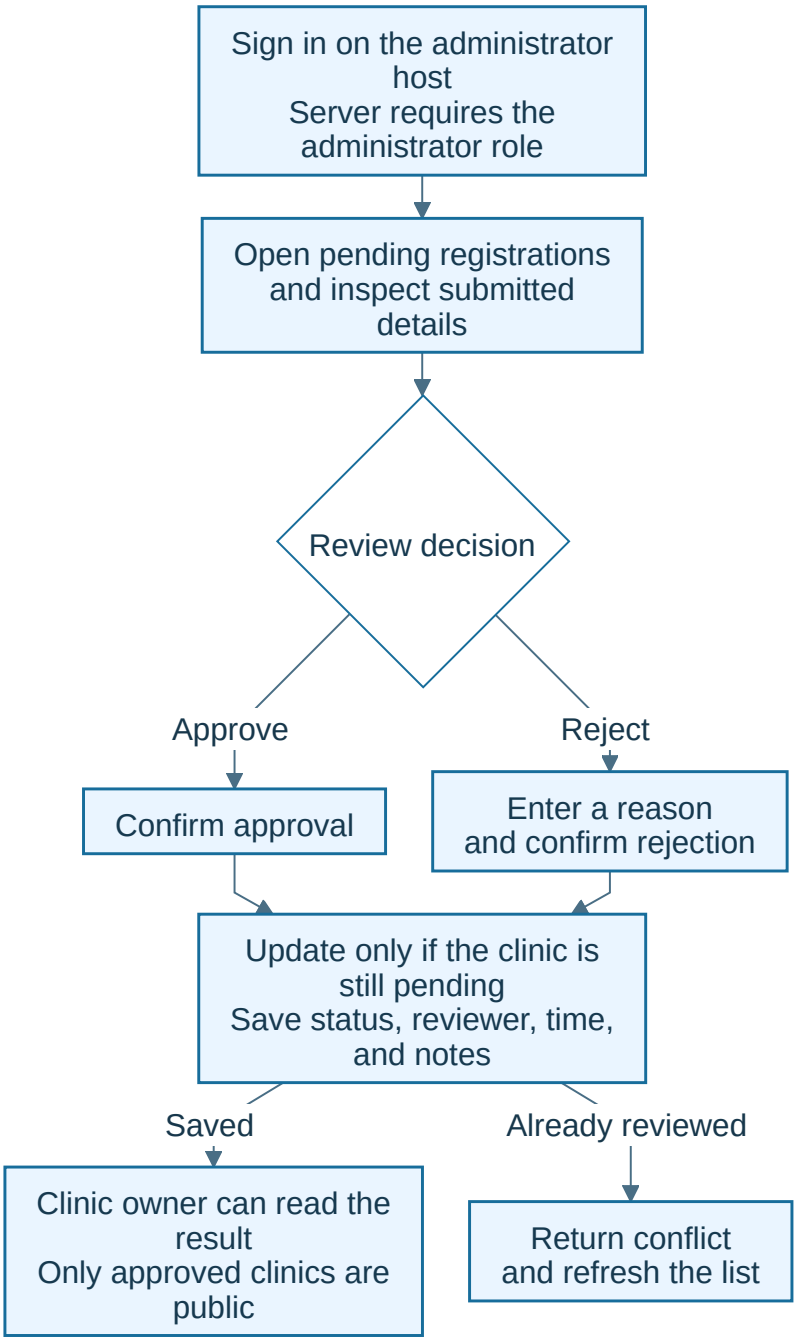

Clinic registration and access — Current workflow · Open the image at full size to zoom.

Registration and staff-account creation are one database transaction. Signing in while pending does not grant approved-clinic access. A suspended status can also be displayed, but a complete suspension-management interface is not delivered.

Open full-size SVG · Editable Mermaid source

## Administrator review process

**Status: Current workflow.**



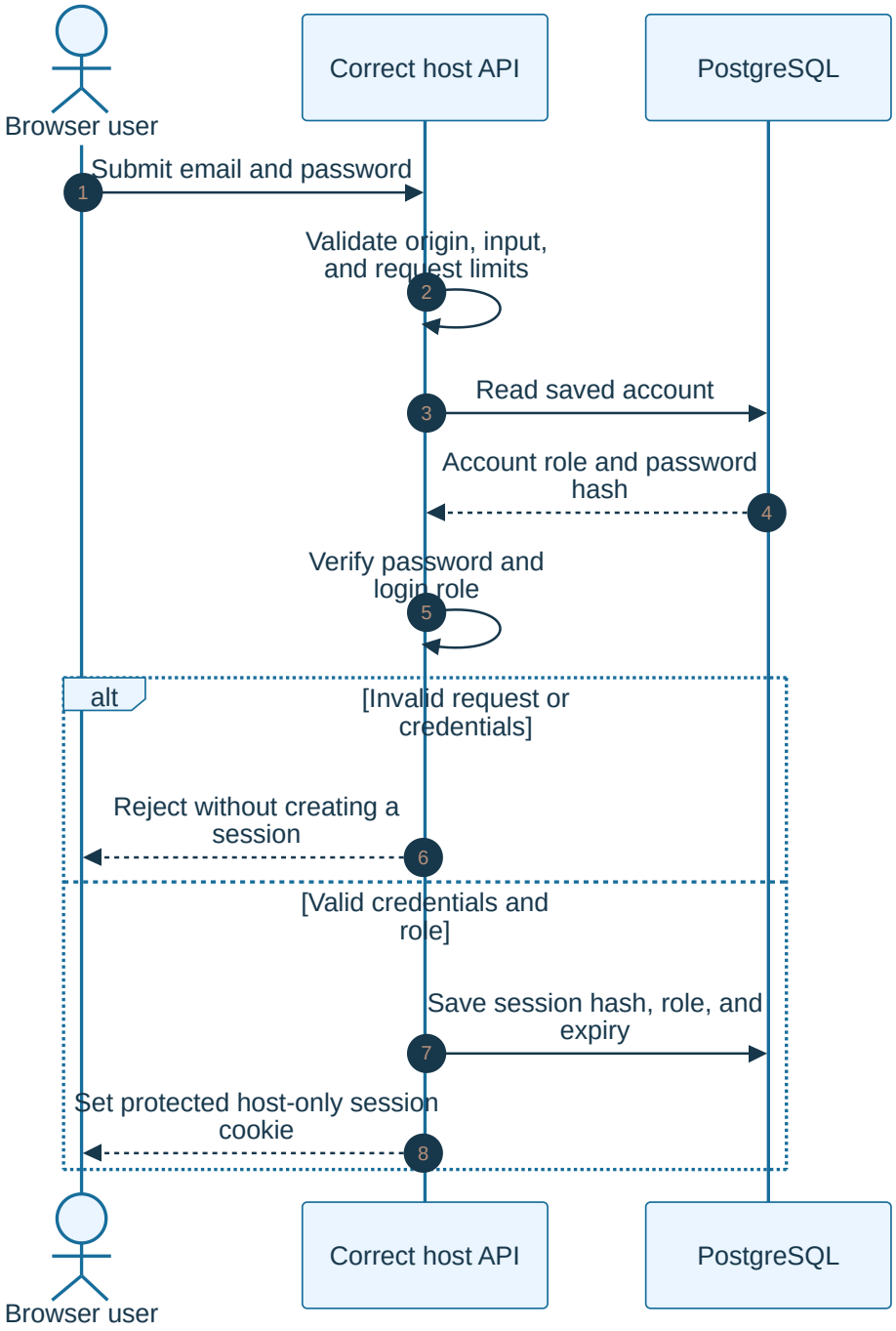

Administrator review process — Current workflow · Open the image at full size to zoom.

The review API permits only pending-to-approved or pending-to-rejected changes. The software records a publishing decision; professional-license verification remains an administrator responsibility.

Open full-size SVG · Editable Mermaid source

## Sign-in and session creation

**Status: Current sequence.**



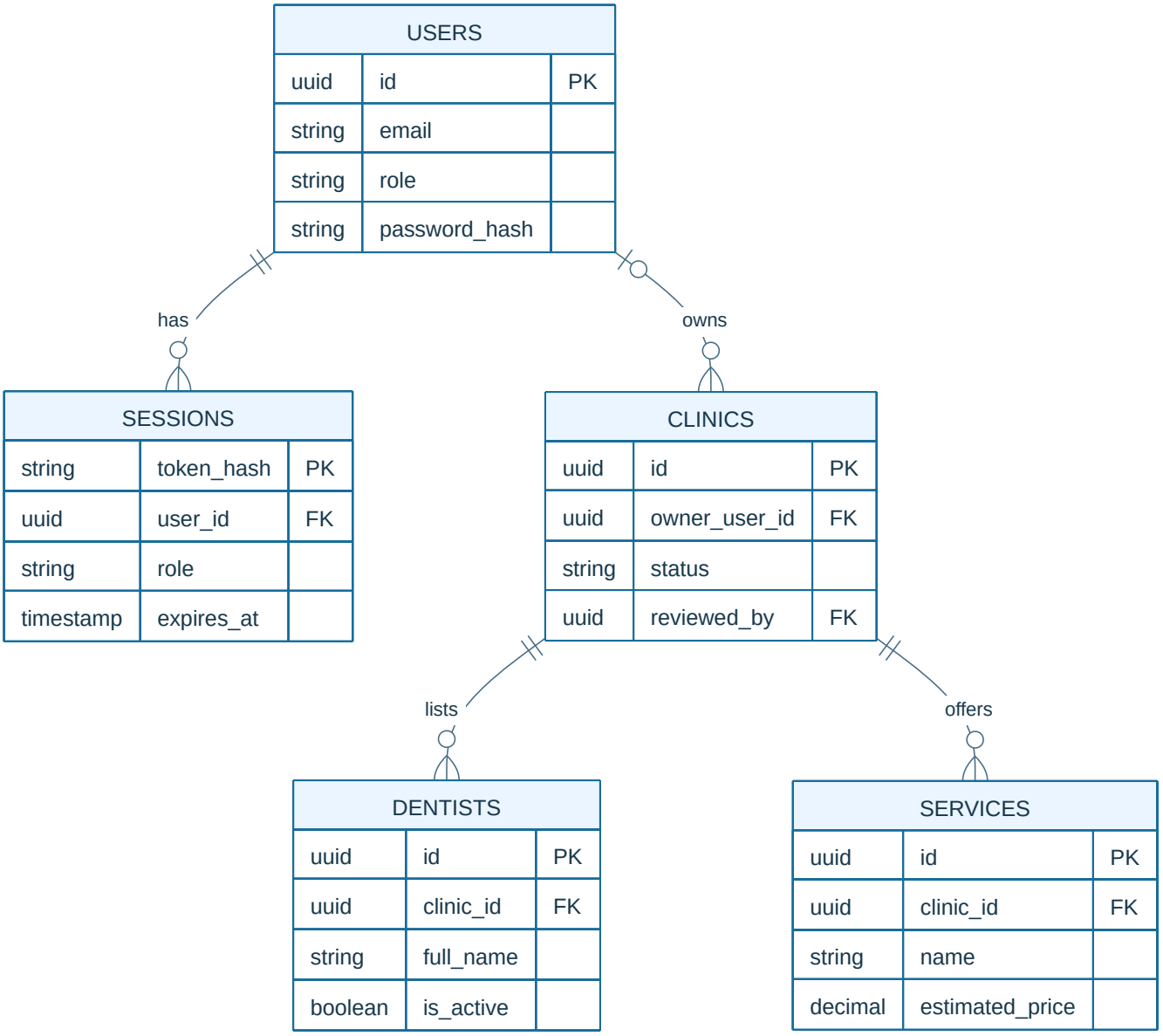
Sign-in and session creation — Current sequence · Open the image at full size to zoom.

Only the hash of the random session token is stored. Administrator sessions last one hour; patient and clinic sessions last eight hours. Logout, expiry, password-hash changes, or role changes revoke access as described in the security guide.

Open full-size SVG · Editable Mermaid source

## Accounts, clinics, and public data

**Status: Current schema.**



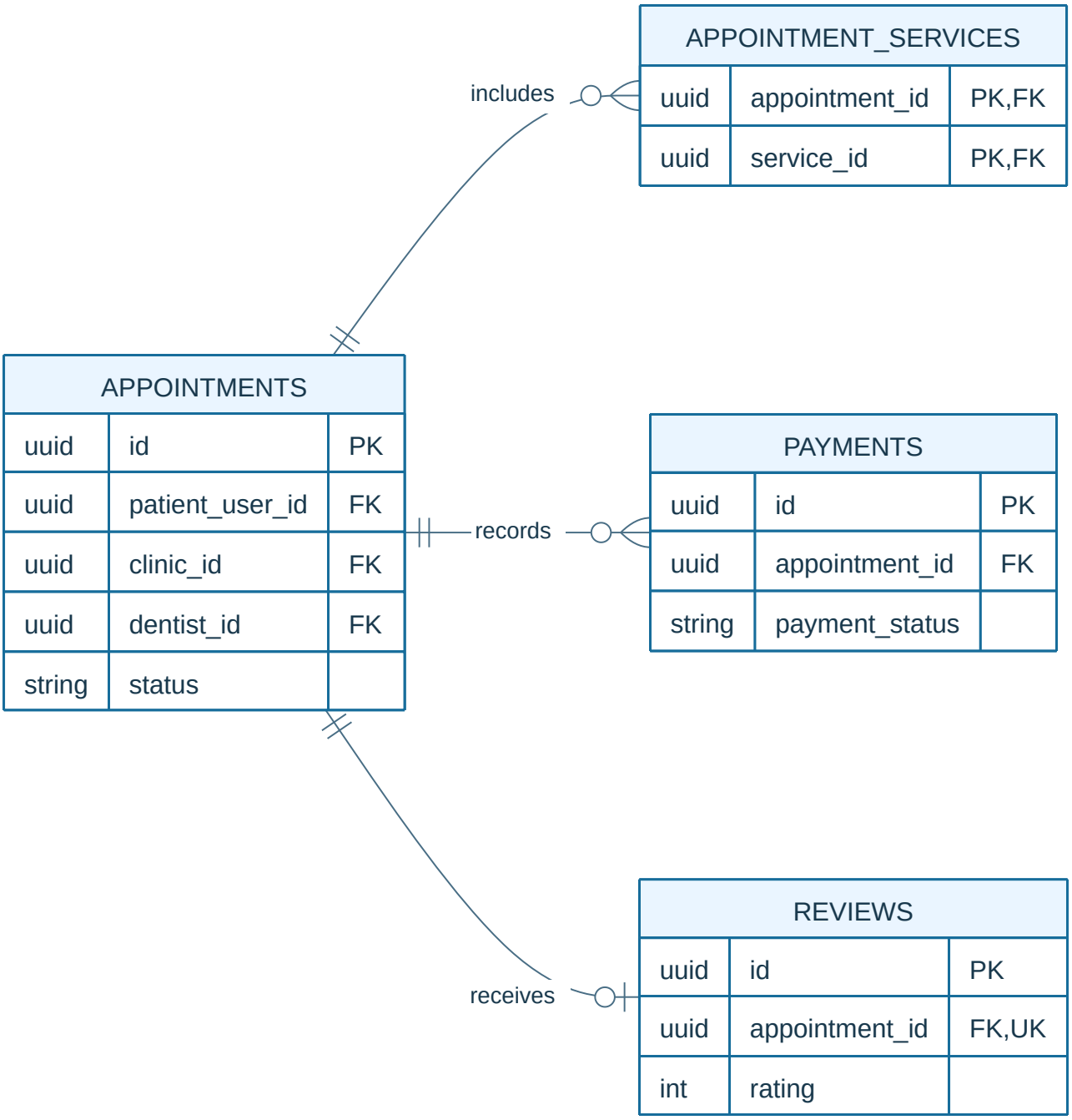

Accounts, clinics, and public data — Current schema · Open the image at full size to zoom.

This is a simplified view of actual foreign keys, not every column. Clinic ownership may be empty at database level; workspace access still requires the signed-in owner and approval. reviewed_by also refers to a user. Public queries filter approved clinics and active dentist/service records.

Open full-size SVG · Editable Mermaid source

## Appointment and payment data foundations

**Status: Existing schema; management workflows planned.**



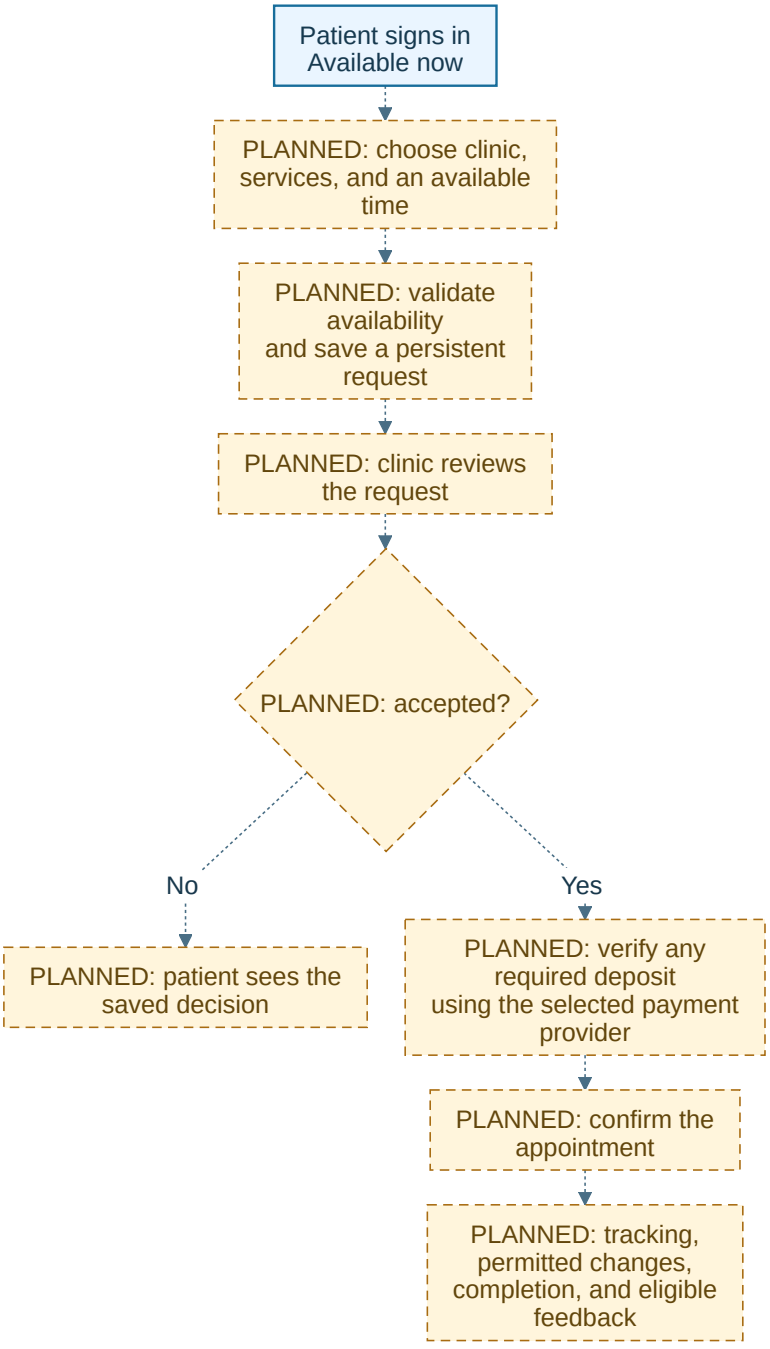

Appointment and payment data foundations — Existing schema; management workflows planned · Open the image at full size to zoom.

The tables already exist, but they do not establish a working booking or payment feature. Only reading existing patient-linked appointments is available. Appointment creation, payment integration, and review submission remain planned. The appointment foreign keys point to users, clinics, and dentists; the service-line foreign key points to services, shown in the preceding core-data diagram. Those parent entities and scheduling/assignment tables are omitted here for readability; see the schema for the full structure.

Open full-size SVG · Editable Mermaid source

## Future appointment workflow

**Status: Planned; not a live booking process.**



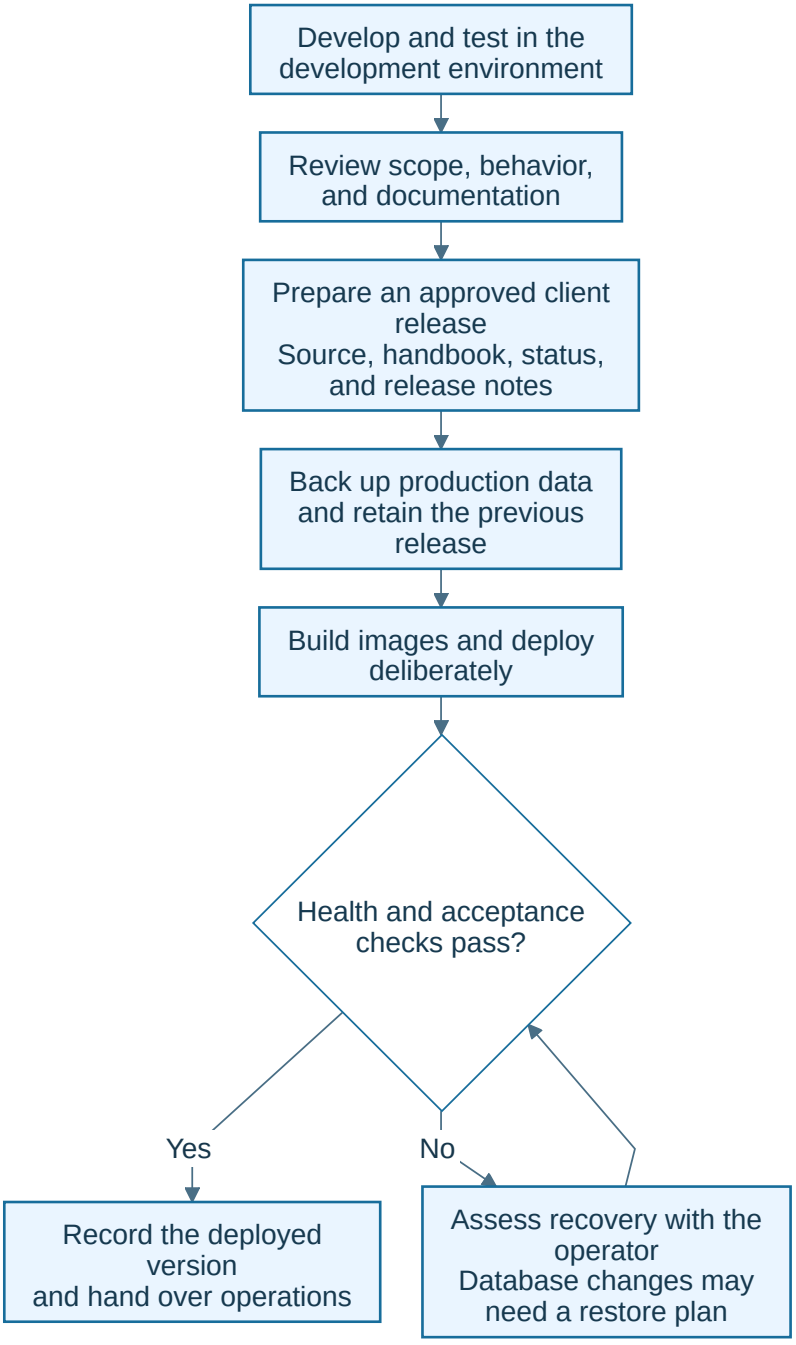

Future appointment workflow — Planned; not a live booking process · Open the image at full size to zoom.

Dashed amber steps are proposed development, not available features. The client must agree scheduling, deposits, cancellation, notification, and feedback rules before implementation. Signing in today does not create a request or collect payment.

Open full-size SVG · Editable Mermaid source

## Reviewed releases and deployment

**Status: Current manual operating process.**



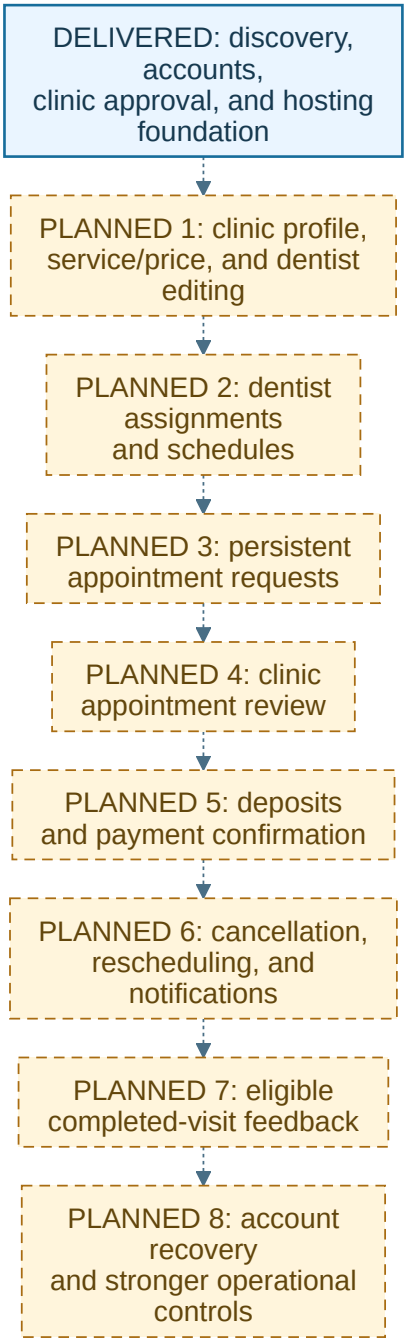

Reviewed releases and deployment — Current manual operating process · Open the image at full size to zoom.

This is an operator-led process, not automatic deployment. Development changes do not update production or the client repository by themselves. Database migrations have no general automatic down command; recovery must account for both code and data.

Open full-size SVG · Editable Mermaid source

## Remaining development phases

**Status: Foundation delivered; next phases planned.**



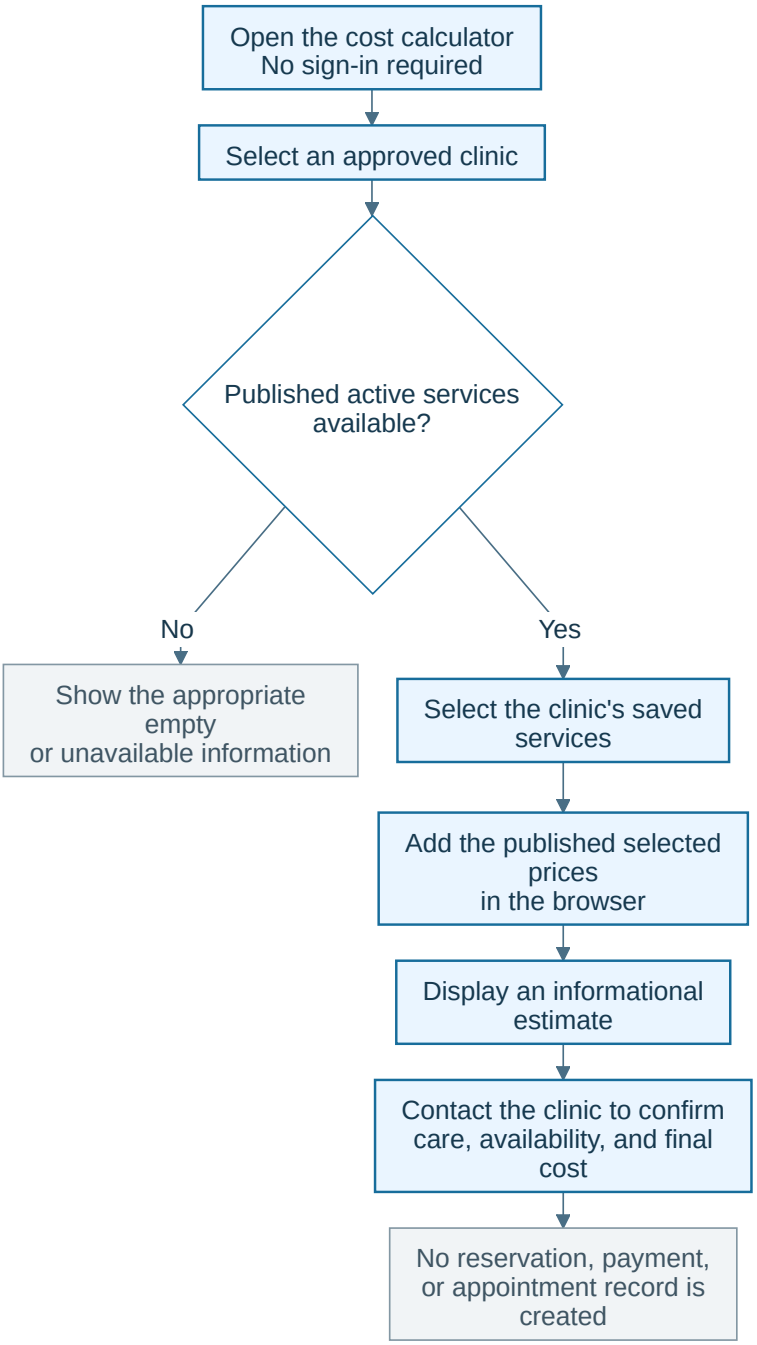

Remaining development phases — Foundation delivered; next phases planned · Open the image at full size to zoom.

The order follows the development plan. These are proposed release phases, not start dates or a claim that work has already begun. Operational responsibilities and backups apply throughout, even where future automation appears later in the roadmap.

Open full-size SVG · Editable Mermaid source

## Published-price cost estimation

**Status: Current workflow.**



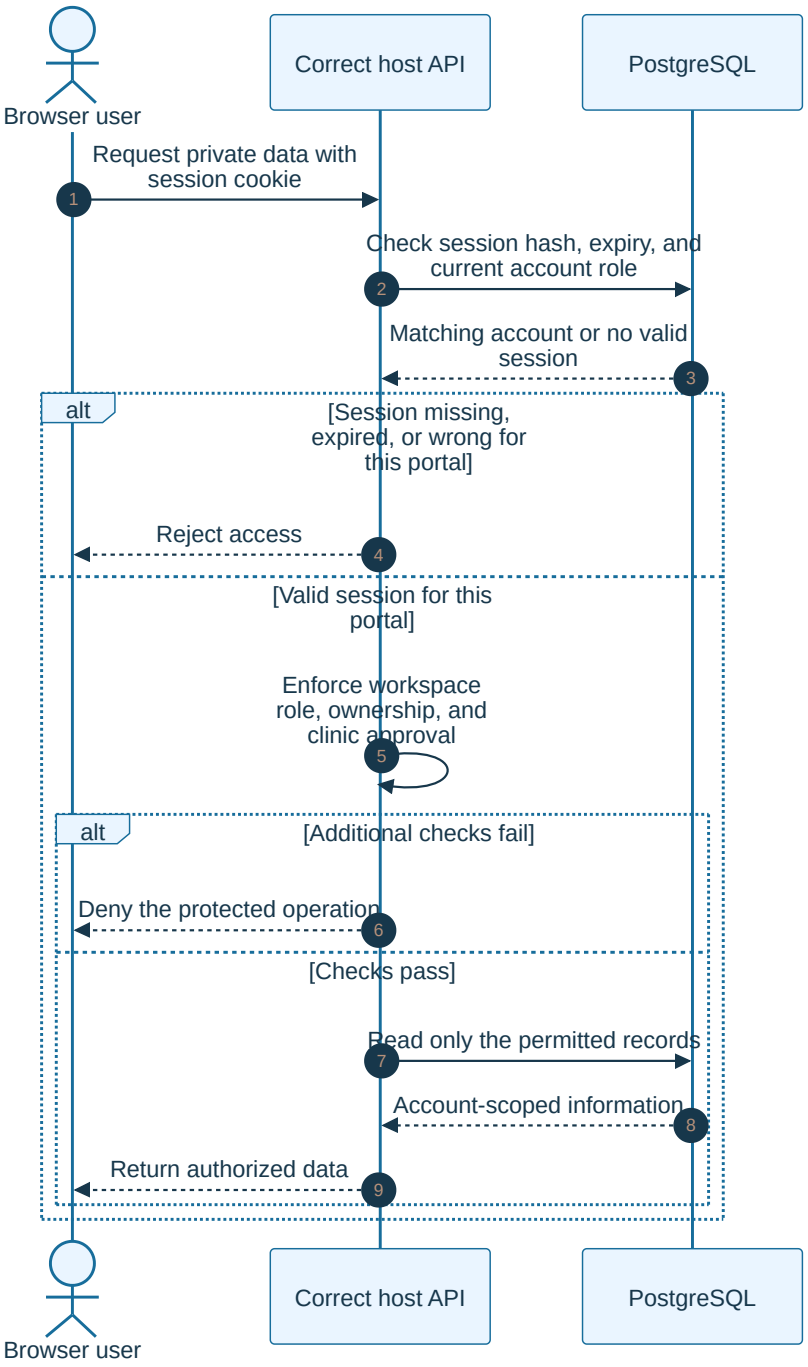

Published-price cost estimation — Current workflow · Open the image at full size to zoom.

The calculator reads published database prices but performs the informational total in the browser. It is not a payment amount authorization or a final treatment quote.

Open full-size SVG · Editable Mermaid source

## Private request permissions

**Status: Current sequence.**




Private request permissions — current sequence · Open the image at full size to zoom.

A valid cookie alone does not authorize every action. The API checks session expiry, account role, the configured public/admin process, and the workspace's rules. Patient records are restricted to the signed-in patient; clinic information is restricted to the owner and approval status. Related database reads needed for approval and ownership are grouped within the permission checks in this overview.

Open full-size SVG · Editable Mermaid source

# Dependency versions

These versions come from the delivered lockfiles. Use `npm ci` to reproduce them. Node.js 22 and PostgreSQL 16 are selected by the Dockerfiles and Compose configuration.

## Website

| Package | Locked version | Use | Purpose |
|---|---|---|---|
| `lucide-react` | 1.64.0 | Runtime | Icons |
| `react` | 19.3.1 | Runtime | Interface components |
| `react-dom` | 19.3.1 | Runtime | Browser rendering |
| `react-router-dom` | 7.18.5 | Runtime | Page navigation |
| `@playwright/test` | 1.64.1 | Build/test | Browser and responsive-layout tests |
| `@vitejs/plugin-react` | 5.2.0 | Build/test | React build integration |
| `vite` | 6.4.5 | Build/test | Frontend builds and development server |

## API

| Package | Locked version | Use | Purpose |
|---|---|---|---|
| `bcrypt` | 6.0.0 | Runtime | Password hashing |
| `cors` | 2.8.6 | Runtime | Allowed browser-origin handling |
| `dotenv` | 17.4.2 | Runtime | Private environment configuration |
| `express` | 4.22.3 | Runtime | HTTP API framework |
| `pg` | 8.23.1 | Runtime | PostgreSQL connection pool |
| `@electric-sql/pglite` | 0.5.13 | Build/test | Isolated PostgreSQL-compatible integration tests |

Nginx and base container images currently use the tags declared in the Dockerfiles. Tags can resolve to newer image builds; record image digests with each deployment if an exact container rebuild is required. The lockfiles cover JavaScript package versions, not container image immutability.

# Branding, photography, and font notices

The DentaBook logo is supplied project branding and is stored in `client/public/branding/dentabook-logo.jpg`.

Illustrative photographs are stored locally in `client/public/images`. They are not presented as evidence of an actual listed clinic, practitioner, professional qualification, or customer endorsement. The homepage uses `hero.jpg`; the About page uses `clinic-1.jpg`. The other included photographs are retained source assets and are not current directory records.

| Asset | Original image source |
|---|---|
| `hero.jpg` | https://images.unsplash.com/photo-1606811841689-23dfddce3e95 |
| `clinic-1.jpg` | https://images.unsplash.com/photo-1629909613654-28e377c37b09 |
| `clinic-2.jpg` | https://images.unsplash.com/photo-1629909615184-74f495363b67 |
| `clinic-3.jpg` | https://images.unsplash.com/photo-1629909615957-be38d48fbbe6 |
| `person-1.jpg` | https://images.unsplash.com/photo-1559839734-2b71ea197ec2 |
| `person-2.jpg` | https://images.unsplash.com/photo-1612349317150-e413f6a5b16d |
| `person-3.jpg` | https://images.unsplash.com/photo-1594824476967-48c8b964273f |

The font files and their supplied license notices are included together:

- `client/public/fonts/dm-sans.ttf` and `dm-sans-license.txt`.
- `client/public/fonts/manrope.ttf` and `manrope-license.txt`.

Dependency versions are listed in dependency versions. Package-specific notices are provided with the corresponding installed packages. Keep supplied notices with the assets when redistributing the project.

## Workflow and technical diagrams

Vector diagrams are supplied under `docs/diagrams/`, with their editable Mermaid source files in `docs/diagrams/src/`. See the diagram guide for their current/planned status and the diagram README for update instructions. The SVG files are pre-rendered documentation assets; Mermaid is not a website runtime dependency.

# Delivery notes

## Workflow and system diagrams — 31 October 2026

Added 12 diagrams covering the system architecture, patient/clinic/admin journeys, session checks, account/clinic data, appointment data foundations, planned booking, cost estimates, release operations, and the remaining roadmap. Each diagram includes a vector SVG and editable Mermaid source. Current behavior and planned features are labeled separately. The diagram guide and printable handbook now include these visuals.

## Initial client handoff — 31 October 2026

This snapshot includes:

- Public clinic discovery and profiles backed by approved database records.
- Treatment information and clinic-price cost estimation.
- Patient accounts and protected personal pages.
- Clinic registration, registration status, and approved-clinic workspace access.
- Administrator clinic review through a separate administration host.
- Database migrations, account-role checks, secure sessions, and origin checks.
- Docker deployment configuration and automated backend/browser checks.
- User, technical, operations, and remaining-development documentation.

The status and development plan is the reference for unfinished product features. This handoff does not represent completion of the full appointment and payment workflow.